

Bob — Design Spike #38.0: Business (multi-employee) accounts data model

Status: decisions agreed (see §0) — build deferred to 2027 · **Date:** 2026-07-25, decisions 2026-07-26 · **Owner:** André **Why now:** #10 (pro verification) ships Sep–Oct 2026 and #12 (Stripe billing) Oct–Dec 2026. Both will bake single-professional assumptions into schema, verification and billing. Settling the org model *on paper* before then avoids a costly 2027 refactor. This doc is a decision aid, not a migration to apply.

Research note: the references in §9 were retrieved and read from the live docs on 2026-07-25 (AWS SaaS Lens, Supabase RLS + RBAC, Clerk Organizations, Stripe Billing). Links are current as of that date. Sources named without a link (NIST RBAC, field-service SaaS, marketplace practice) are well-established and stated from knowledge.

0. Decision sheet (agreed 2026-07-26)

The direction chosen is **Bundle 2 — "clean org model", built additively**: adopt the separate-entities model now (organization / member / public seller profile as distinct things), but ship it as nullable additions so solo pros are untouched. The value is timing — #10 (verification) and #12 (billing) get built org-aware rather than reworked in 2027.

#	Decision	Choice	Why
1	Workers: records or users	Records , scoped login as a later step	Scheduling correctness never depended on worker auth (§6.1)
2	Booking / assignment	Company assigns the worker	Keeps Bob on the "employer allocates" side of AI-Act/Platform-Work scope (§2, §4)
3	Overlap enforcement	Database constraint , not app-only	Multi-worker + concurrent bookings is where app-level checks leak (§7)
4	Reviews	Company-level	Simpler, protects workers, fits trust-first positioning (§4 D1)
5	Communication identity	Company identity + worker attribution ("Mario, [Company]")	Sidesteps per-worker reputation and PWD consequences (§6.2)
6	Roles	Enum <code>owner\ admin\ worker</code>	Upgrade to the two-table RBAC pattern only if permissions proliferate (§4 B)
7	Tenant isolation	Pool model + one <code>is_org_member()</code> helper	<code>select</code> -wrapped and indexed, per Supabase RLS guidance (§4 F, §9-B)
8	Billing	Per-seat via Stripe <code>quantity</code> ; flat-rate acceptable for pilot if #12 timeline is at risk	The one place to trade purity for schedule (§4 E, §9-F)

Deliberately left open: the timing and form of step-2 worker access (per-job magic link vs limited login). This depends on how much real-time customer contact the first business accounts actually need — a pilot question, not a design question. Do not pre-commit.

Immediate next action (no build): validate these answers against #10 and #12 before those start — specifically (a) can verification handle a *company* (visura / P.IVA) as well as a person, and (b) can the Stripe model carry a seat `quantity` later without a migration?

1. Where Bob is today (from the live schema)

- `professionals` is **1:1 with** `users` (`professionals.user_id` is UNIQUE). One person = one professional.
- Everything hangs off a single `professional_id`: `appointments`, `professional_availability`, `professional_availability_blocks`, `professional_services`, `ratings`, `portfolio_items`, `subscriptions`, `payments`, `payouts`, `pro_payment_accounts`.
- `professionals.subscription_tier` ∈ (free, pro, business) already exists — but per the business plan "Business" is today a **premium single-pro plan** (e-invoicing, payments, analytics, unlimited portfolio), *not* a company with staff.
- `users.role` ∈ (customer, professional, admin, cs) — no org/worker concept.

So the gap is real: there is no company entity, no membership/worker entity, and no way to assign an appointment to a specific worker.

2. What the model must satisfy

1. A **company entity** with its own legal/billing identity and documents (VAT / visura), distinct from any one person.
 2. **Multiple members** with roles (at least owner / admin / worker; maybe dispatcher).
 3. Appointments **assignable to a specific worker**, with **per-worker availability**.
 4. A clear answer to: does the client book *the company* or *a named worker*?
 5. **Per-company billing**, most likely **per-seat**.
 6. **Backward compatibility**: solo pros are the common case at launch and must keep working untouched.
 7. **Compliance** (per `DATA_COMPLIANCE.md`): the employer is the **controller** of its employees' data, Bob is the tooling provider (**processor**, possibly joint); keep the **"employer assigns the worker"** model (never Bob auto-allocating) to stay out of AI-Act Annex III / Platform Work Directive scope; RLS on every new table; retention + deletion path for member records.
-

3. The one fork that changes everything: are workers *users* or *records*?

Decide this first — it drives auth, RLS and GDPR more than any other choice.

- **Workers as records (no login).** The business owner creates worker rows and manages their calendars. Bob never authenticates the worker. → Cleanest GDPR story: business is unambiguously the controller, Bob a processor; no new auth surface. Matches most field-service SaaS (a dispatcher schedules technicians who may not log in). Limit: workers can't self-serve their own calendar or chat with clients.
- **Workers as users (own login).** Each worker is a Bob `user` linked to the org. → Enables worker self-service, per-worker chat, mobile use. Cost: more auth/RLS complexity, and Bob processes more personal data of the worker directly (access requests, deletion, etc.).

Suggested answer: start **workers-as-records** for v1 (fast, clean compliance), with the schema shaped so a worker record can later be *linked* to a real user without a rewrite (nullable `user_id` on the worker/member row). This is the "progressive" principle already in Bob's business plan.

4. Decision dimensions (each has options; pick one per dimension)

A. How an organization relates to `professionals`

- **A1 — Org as optional owner (additive).** Add nullable `organization_id` to `professionals`; a solo pro is a `professionals` row with `organization_id = NULL`. Smallest change. Risk: the `professionals` row keeps conflating "a person" and "a company profile."
- **A2 — Org *is* the professional.** The `professionals` row represents the business; workers are members who fulfil. Conceptually clean for companies but awkward for solo pros and a bigger migration.
- **A3 — Separate worker entity.** `professionals` = the public *seller/listing* (owned by either a solo user or an org); a distinct `organization_members` / `workers` table holds fulfilment staff; appointments reference a worker. Most flexible, clearest separation of "who is listed/sells" vs "who does the job."

Literature: this is the classic multi-tenant "**pool**" / **shared-schema-with-tenant-id** model (Microsoft Azure SaaS tenancy patterns; AWS SaaS "silo / pool / bridge"). The org ↔ membership ↔ user triangle is the standard B2B identity shape used by WorkOS, Auth0 Organizations and Clerk Organizations.

B. Membership & roles (RBAC)

- **B1 — Enum role on membership** (`role ∈ owner|admin|worker`). Enough for Bob's scale.
- **B2 — Full RBAC** (separate `roles` + `permissions` join tables). More future-proof, more machinery.

Literature: NIST RBAC model (Sandhu et al., 1996; ANSI INCITS 359-2004). Supabase's own "Custom Claims & RBAC" guidance implements B1/B2 via a membership table + JWT claims. **B1 is the right default**; upgrade to B2 only if permissions proliferate.

C. Appointment assignment & scheduling

- **C1** — `appointments.worker_id` nullable + per-worker availability. Employer assigns; unassigned = "any/house."
- **C2** — **Org-level appointments + separate assignment table.** More normalized, more joins.

Literature: field-service SaaS (Jobber, Housecall Pro, ServiceTitan) all use a **dispatch** model — a scheduler assigns jobs to technicians. Crucially the **employer dispatches internally**; Bob must not auto-allocate, or it edges into AI-Act Annex III 4(b) / Platform Work Directive 2024/2831 (see `DATA_COMPLIANCE.md §2`). **C1** is simplest and keeps the compliant "employer assigns" posture.

D. Public identity & reviews

- **D1** — **Company-level.** Client books and reviews the company; workers are internal. Simpler, protects worker privacy, matches how clients think about a firm.
- **D2** — **Worker-level.** Client picks/reviews a named worker. Richer, but multiplies reputation cold-start and exposes individual workers publicly.

Literature: platform reputation research and marketplace practice split here — Thumbtack/Yelp review the **business**; Uber/TaskRabbit rate the **individual**. For Bob's trust-first, anti-lavoro-nero positioning, **D1 (company-level)** is the safer default; revisit D2 only if clients demand specific-worker choice (which itself pushes toward Platform-Work-Directive scrutiny).

E. Billing

- **E1** — **Flat Business tier per org** (keep 59€/49€). Simplest; ignores headcount.
- **E2** — **Per-seat** (base + N×worker). Aligns price with value; standard SaaS.
- **E3** — **Seat bands** (e.g. 1–3, 4–10). Middle ground, fewer proration headaches.

Literature: Stripe distinguishes **licensed** (fixed quantity = seats) vs **metered** billing; per-seat is the dominant B2B SaaS model. **E2/E3** fit a multi-worker product; **E1** is fine for a v1 pilot to avoid proration complexity, upgrading later.

F. Tenant isolation / RLS

- **F1** — `organization_id` column + **RLS via a membership lookup** (pool model, shared schema). One helper, e.g. `is_org_member(org_id, min_role)`, referenced by every org-scoped policy.
- (Silo / database-per-tenant is **not** appropriate at Bob's scale — operational overkill.)

Literature: Supabase RLS multi-tenancy guidance (tenant_id + policies using `auth.uid()` / JWT claims); AWS "pool" isolation; Azure "single multi-tenant DB, row-level isolation." **F1 is the only sensible choice here** and matches Bob's existing RLS-on-every-table rule.

5. Three coherent bundles (so the choice is a whole, not 6 loose knobs)

Bundle	Dimensions	Effort	When it's right
1 — Minimal / additive	A1 + B1 + C1 + D1 + E1 + F1; workers-as-records	Low (days)	Ship business accounts fast for the pilot; accept some conceptual debt in professionals
2 — Clean org model (recommended)	A3 + B1 + C1 + D1 + E2/E3 + F1; workers-as-records now, <code>user_id</code> -ready	Medium (1–2 wk schema + UI)	Business accounts are strategic; you want a model that lasts without another migration
3 — Full RBAC platform	A3 + B2 + C2 + D2 + E2 + F1; workers-as-users	High	Only if Bob becomes a serious field-management tool; over-engineered for now

6. Recommendation

Adopt **Bundle 2's model on paper now, implement additively**:

1. New `organizations` (name, VAT/P.IVA, docs, billing owner, created_at) — nullable link so **solo pros are untouched**.
2. New `organization_members` (`org_id`, nullable `user_id`, `full_name`, `role` owner|admin|worker, status, created_at) — starts as **records**, upgradeable to users.
3. Treat `professionals` as the **public seller profile**, owned by *either* a solo user *or* an org (add nullable `organization_id`; keep the existing solo path).
4. `appointments.worker_member_id` nullable → per-worker assignment by the employer.
5. Per-worker availability: add nullable `member_id` to `professional_availability` (+ blocks), or a parallel table.
6. Billing per-seat via **Stripe quantities** (E2/E3), extending #12 rather than replacing it.
7. RLS: pool model (**F1**) with one `is_org_member(org_id, min_role)` helper reused everywhere.

Why this and not Bundle 1: the additive parts are the *same* either way, but committing to the **separate-entities mental model** (A3) now means #10 verification and #12 billing can be built "org-aware" from day one instead of being reworked in 2027. It stays inside Bob's rules — a migration file per change, RLS on every table, employer-as-controller, employer-assigns.

Why not Bundle 3: full RBAC and worker-as-user auth are real cost with no pilot-stage payoff; both are reachable later without a rewrite because the schema leaves `user_id` and role open.

6.1 Scheduling with record-workers

The concern "if workers have no login, how does Bob manage calendars and overlaps?" dissolves once you see that **records-vs-users is a decision about login/identity, not about whether the scheduling data exists**. Overlap detection, availability and calendar management all live in the data layer and depend on the worker having a *row*, not a *session*. This is the field-service dispatcher model

(Jobber / Housecall Pro / ServiceTitan): a logged-in owner/admin manages the calendars; the technician need never log in for the schedule to be correct.

- **Availability** is keyed to `member_id`: each worker record has its own weekly hours (`professional_availability`) and time-off ranges (`professional_availability_blocks`), edited by the owner/admin on the worker's behalf.
- **Appointments** carry `worker_member_id`; an assigned appointment occupies that worker's calendar.
- **Overlap / double-booking** reuses the guard already built in 8.7a — re-keyed from `professional_id` to `worker_member_id` so the check runs per worker. Enforce it at the database with a concurrency-safe Postgres exclusion constraint (see §7) so two overlapping appointments for the same worker are simply impossible, even under simultaneous bookings.
- **Free-slot engine, per worker**: bookable slots = availability windows – time-off blocks – existing appointments. When a client books "the company," compute org availability as the **union** of workers' free slots, then assign a free worker at confirm time (the exclusion constraint guarantees the assignment is conflict-free).
- **Booking granularity** is a product choice: client picks a worker, or the company assigns one internally. Company-assigns keeps Bob cleanly on the "employer allocates" side of the compliance line.

None of this requires the worker to authenticate. It all runs off the owner/admin session plus server-side logic.

6.2 Communication model — step 1 → step 2

Communication is where the records model reaches its limit, so separate two things that are usually conflated:

- **Attribution** — whose name the customer sees on a message. A message can display "Mario, [Company]" via a `sender_member_id` (the worker record) on `request_messages`.
- **Authorship** — who actually pressed send, i.e. the authenticated `sender_id`.

These are independent, which is what makes the staged plan work.

Step 1 (records only): the office is the communication hub. The assigned worker has no login, so anything touching the customer is done by the logged-in owner/admin/dispatcher on the company's behalf, and can still be *attributed* to the worker.

Use case — worker running 30 min late: worker tells the office → dispatcher opens the appointment in Bob, messages the customer ("Mario is running ~30 min late") and/or proposes a new slot from the worker's free slots → customer confirms in-app. The free-slot engine and overlap constraint keep the reschedule conflict-free.

This keeps everything on-platform but inserts a human relay — impractical for real-time and annoying at scale. That friction is precisely the signal that a worker needs their own channel.

Step 2 (scoped worker access): the worker communicates directly. The moment a worker must contact the customer or change an appointment *themselves*, they need to act on Bob — which is authentication. It need not be a full account:

- *Per-job magic link* (lightest) — an SMS/email link to just their assigned job: read details, message the customer in the existing thread, propose a reschedule. No password, scoped to that appointment. Common field-service pattern.
- *Full limited-role login* — the worker becomes a real `user` linked to their record via the nullable `user_id`, role `worker`, with RLS restricting them to their own assigned jobs and threads.

Either way it is **additive**: the thread already exists and attribution already supports `member_id`, so step 2 only adds an authenticated actor plus a read/write RLS scope — no data migration.

Guardrails (from `DATA_COMPLIANCE.md`):

- Keep the primary channel **in-app, not a phone hand-off** — giving the worker the customer's number triggers the anti-disintermediation and independent-controller problems (§4). Progressive disclosure still applies (full contact only after the customer accepts); the Pro ToS forbids reusing contacts. Masked/in-app calling can come later if physical jobs need voice.
- **Worker RLS is least-privilege** — a worker sees only their assigned appointments and those threads, never the whole company's book.

Open decision (drives schema): in step 2, does the customer see and message an **individual worker** ("Mario") or always **the company** (worker replies under the company name)? Company-name-with-attribution is simpler and cleaner on the compliance line; per-worker identity is richer but pushes toward per-worker reviews and more Platform-Work-Directive scrutiny.

7. Migration sketch (illustrative — to validate, not to apply)

```

-- 038_business_orgs.sql (idempotent sketch)
create table if not exists organizations (
    id uuid primary key default extensions.uuid_generate_v4(),
    name text not null,
    vat_number text,          -- P.IVA; verification extends #10
    billing_owner_user_id uuid references users(id),
    status text not null default 'active' check (status in ('active','suspended')),
    created_at timestamptz not null default now()
);

create table if not exists organization_members (
    id uuid primary key default extensions.uuid_generate_v4(),
    organization_id uuid not null references organizations(id) on delete cascade,
    user_id uuid references users(id),          -- NULL = record-only worker
    full_name text not null,
    role text not null default 'worker' check (role in ('owner','admin','worker')),
    status text not null default 'active',
    created_at timestamptz not null default now()
);

alter table professionals add column if not exists organization_id uuid references organizations(id);
alter table appointments add column if not exists worker_member_id uuid references organization_members;
alter table professional_availability add column if not exists member_id uuid references organization_members;
alter table professional_availability_blocks add column if not exists member_id uuid references organization_members;

-- messaging attribution: whose name shows (member) vs who sent (sender_id), independent
alter table request_messages add column if not exists sender_member_id uuid references organization_members;

-- concurrency-safe overlap guard: no two proposed/confirmed appointments overlap for the same worker
create extension if not exists btree_gist;
alter table appointments drop constraint if exists no_worker_overlap;
alter table appointments add constraint no_worker_overlap
exclude using gist (
    worker_member_id with =,
    tstzrange(starts_at, starts_at + (duration_minutes || ' min')::interval) with &&
) where (worker_member_id is not null and status in ('proposed','confirmed'));

-- RLS helper (pool model)
create or replace function is_org_member(p_org uuid, p_min_role text default 'worker')
returns boolean language sql stable security definer set search_path = public as $$
select exists (
    select 1 from organization_members m
    where m.organization_id = p_org and m.user_id = auth.uid()
    and (p_min_role = 'worker'
        or (p_min_role = 'admin' and m.role in ('admin','owner')))
);

```

```

        or (p_min_role = 'owner' and m.role = 'owner'))
    );
$$;

alter table organizations enable row level security;
alter table organization_members enable row level security;
-- policies: members read own org; only owner/admin write; service_role bypass.
-- retention/deletion: on member removal, de-identify historical appointments (SET NULL + placeholder), r

```

After any real version of this: run the Supabase security advisors and fix RLS-missing / SECURITY DEFINER / mutable search_path findings, per the standing rule.

8. Question status

Question	Status
Book the company or a named worker?	Closed — company assigns the worker (§0.2)
Workers = users or records for v1?	Closed — records, scoped access later (§0.1, §6.2)
Reviews company-level or per-worker?	Closed — company-level (§0.4)
Billing flat, per-seat, or bands?	Closed — per-seat via Stripe <code>quantity</code> ; flat acceptable for pilot (§0.8)
Verification: company docs vs per-worker ID?	Open — to settle against #10; likely company-level (visura/P.IVA) + optional per-worker
Do solo pros ever become orgs?	Open — confirm upgrade flow (create org, attach existing <code>professionals</code> row)
Step-2 worker access: magic link or login, and when?	Deliberately open — pilot-driven (#38.6); do not pre-commit
Step-2 message identity: individual worker or company?	Open — leaning company-name-with-attribution (§6.2)

9. References

A. Tenant isolation / how the org relates to data → supports dimensions A + F

- AWS Well-Architected, SaaS Lens — *Silo, Pool, and Bridge Models*: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/silo-pool-and-bridge-models.html>. Read 2026-07-25. The **silo** model gives each tenant dedicated resources (e.g. a separate database); the **pool** model has tenants share scalable infrastructure — "the more classic notion of multi-tenancy"; **bridge** mixes both per microservice. For Bob at pilot scale the pool model (shared schema + `organization_id` + RLS) is the clear fit; silo/db-per-tenant is operational overkill.
- Microsoft Azure SQL — *Multitenant SaaS patterns*: <https://learn.microsoft.com/en-us/azure/azure-sql/database/saas-tenancy-app-design-patterns>. Same taxonomy from the DB side

(standalone single-tenant, database-per-tenant, sharded multi-tenant, single multi-tenant DB with row-level isolation).

B. RLS enforcement of tenancy → supports F1 + the migration sketch

- Supabase — *Row Level Security*: <https://supabase.com/docs/guides/database/postgres/row-level-security>. Read 2026-07-25. Confirms the membership-lookup pattern the sketch uses — scope a table by `team_id` in `(select team_id from team_user where user_id = (select auth.uid()))`, wrap helper calls in `select` so Postgres caches them per-statement (their benchmark: a `security definer` function policy went from ~178s to ~12ms), always set `TO authenticated`, and index policy columns. Directly validates the `is_org_member()` helper design.

C. Roles / RBAC → supports dimension B (B1 vs B2)

- Supabase — *Custom Claims & Role-Based Access Control (RBAC)*: <https://supabase.com/docs/guides/database/postgres/custom-claims-and-role-based-access-control-rbac>. Read 2026-07-25. Shows the exact two-table shape for B2 (`user_roles` + `role_permissions`) with an `authorize(permission)` function read from a JWT claim set by a Custom Access Token hook. Bob's `organization_members.role` enum is the lighter B1 form of the same idea; this is the upgrade path if permissions proliferate.
- NIST RBAC — Sandhu, Coyne, Feinstein & Youman (1996); ANSI INCITS 359-2004. The formal basis for separating users, roles and permissions.

D. Org ↔ membership ↔ user model → supports dimensions A3 + B1

- Clerk — *Organizations*: <https://clerk.com/docs/guides/organizations/overview>. Read 2026-07-25. Canonical B2B shape — users belong to multiple organizations, each membership carries roles/permissions defined once at app level, with an "active organization" per session and invitation / verified-domain / SSO enrollment paths. Mirrors the proposed `organizations` + `organization_members` design (and the workers-as-records-vs-users choice maps to Clerk's invitation vs record distinction).
- Comparable products: WorkOS Organizations, Auth0 Organizations — same triangle.

E. Appointment assignment / dispatch → supports dimension C1

- Field-service SaaS — Jobber, Housecall Pro, ServiceTitan (product docs). Company-with-technicians + a dispatcher who assigns jobs to workers. The key compliance point: the **employer** dispatches internally, so Bob never auto-allocates.

F. Billing → supports dimension E

- Stripe Billing — subscriptions / pricing models: <https://docs.stripe.com/billing/subscriptions/build-subscriptions>. Read 2026-07-25. Licensed (seat) pricing is billed via `quantity` on the subscription line item (usage-based billing omits `quantity`); provisioning is driven by `checkout.session.completed` + `invoice.paid` webhooks — the same webhook → `subscription_tier` sync already planned in #12. Per-seat = set `quantity` to the active worker count.

G. Reviews: company- vs worker-level → supports dimension D

- Marketplace practice: Thumbtack / Yelp review the **business**; Uber / TaskRabbit rate the **individual**. Bob's trust-first positioning favors company-level (D1).

H. Compliance framing → why "employer assigns, not Bob auto-allocate"

- EU AI Act (Reg. 2024/1689) Annex III 4(b) & recital 57; Platform Work Directive 2024/2831; and Bob's own `DATA_COMPLIANCE.md` §2 (Art 22 / Annex III) and §4 (controller/processor, reviews).